

# Robust AI for Network Intrusion Detection System (NIDS)

- **Team Members:** Robin Miah (ID: 107026), Rafael Aza Mrong
- **Supervised by:** Prof. Dr.-Ing. Marcus Purat

## Introduction & Context

- **What is NIDS?** A Network Intrusion Detection System monitors network traffic to identify malicious activities or policy violations in real time[1]. It serves as a critical defense layer, alerting admins to intrusions, attacks, or malware on the network.
- **Where & Why used?** NIDS are deployed in enterprise networks, data centers, and critical infrastructures to protect against ever-increasing cyber threats[1]. As networks grow in size and complexity, manual monitoring is infeasible – intelligent automated detection is essential.
- **Why is it interesting?** Cyber-attacks continue to rise in sophistication and frequency. Effective IDS improve security by detecting novel attacks that may bypass traditional firewalls. This project explores how robust AI techniques can enhance NIDS effectiveness in detecting intrusions.

## Motivation & Relevance of AI-based NIDS

- **Limitations of Traditional IDS:** Signature-based systems rely on known attack patterns and struggle with new or evolving threats. They also generate many false alarms due to static rules. There is a need for adaptive methods that can detect **unknown or variant attacks**.
- **Why Machine Learning?** ML-based IDS can learn to recognize both known and novel attack behaviors from data[2]. They have demonstrated high success in identifying threats by generalizing from past traffic patterns[3]. AI models can handle large volumes of traffic data and adjust to new attack strategies over time.
- **Real-world Relevance:** With the proliferation of IoT and high-speed networks, human analysts cannot inspect all traffic. An AI-driven NIDS provides **real-time automated threat detection** with higher accuracy and speed, improving an organization's security posture. It addresses the urgent need for advanced, adaptive cybersecurity solutions in modern networks.

## Problem Statement & Objectives

- **Problem:** Current NIDS solutions face challenges such as high false positive rates, difficulty detecting new attack types, and lack of interpretability. Ensuring high detection accuracy while maintaining low false alarms remains difficult in practice. The “robustness” of an AI-based NIDS refers to its reliability across varied conditions (e.g. class imbalance, evolving threats).
- **Objectives:**

- Develop and evaluate several ML models (Random Forest, XGBoost, SVM, MLP) for binary network intrusion detection (normal vs attack) using the CICIDS2017 dataset.
- Achieve high detection performance (maximize accuracy, precision, recall, F1-score) and analyze errors (false positives/negatives).
- Compare the models to identify which approach yields the best accuracy and robustness for NIDS.
- Investigate model behavior and limitations, laying groundwork for future extension to multi-class attack categorization and integration of explainable AI.

## Literature Review (ML & XAI for NIDS)

- **ML in NIDS:** Prior research applies various ML algorithms to IDS data. For example, Hamidou and Adda (2025) compared Random Forest, XGBoost, and DNN on intrusion datasets. Random Forest achieved ~99.8% accuracy, slightly outperforming XGBoost and a deep neural network[4]. These results underscore that ensemble tree methods can be highly effective for intrusion detection.
- **Ensembles & Class Imbalance:** Tree-based ensembles (RF, XGBoost, LightGBM, etc.) often handle imbalance and complex patterns well. In one study using NSL-KDD, class balancing with SMOTE further improved detection rates[5][4]. Similarly, on the newer UNSW-NB15 dataset with multiple attack types, boosted trees (XGBoost/CatBoost) reached about 87% accuracy after addressing class imbalance[6].
- **XAI for IDS:** Recent work emphasizes making IDS models explainable. Mohale and Obagbuwa (2025) integrated XAI techniques (LIME, SHAP, ELI5) with ML-based IDS to improve transparency[6]. Their XAI-enhanced models provided feature-importance insights while maintaining high predictive performance, helping analysts trust and understand alerts.
- **Frameworks:** Muhammad *et al.* (2025) proposed **L-XAIDS**, a framework combining LIME with decision tree surrogates to explain intrusion detection decisions[7]. Such approaches deliver local explanations (for specific detections) and global explanations (overall feature importance), addressing the “black-box” nature of ML models in cybersecurity.

## Dataset: CICIDS2017 & Preprocessing

- **About CICIDS2017:** We used the CICIDS2017 benchmark dataset[8], which contains realistic benign traffic and **seven common attack types** (e.g., DDoS, brute force, infiltration, botnet, web attacks). It was created by the Canadian Institute for Cybersecurity to overcome limitations of older IDS datasets (more diversity, updated attacks, labeled flows)[8]. The full dataset encompasses ~2.83 million network flow records collected over 5 days, with each flow labeled as *Benign* or a specific attack.
- **Data Preparation:** The raw data came in multiple CSV files (one per day/attack scenario). We merged these into a single dataset and **sampled 40,000 flows**

(stratified by class) for manageable experimentation (maintaining ~80% benign, 20% attack ratio). We then split this sample into training, validation, and test sets (roughly 70% train, 15% val, 15% test).

- **Feature Processing:** CICIDS2017 provides ~80 features per flow (e.g., duration, byte counts, packet rates, flags). We dropped non-informative fields (Flow ID, timestamps) and encoded categorical features (protocol, etc.) as needed. For algorithms like SVM and MLP, we applied feature scaling (Standardization) to normalize numeric feature ranges. No significant data leakage issues – training data was separated in time from test data to emulate real deployment.
- **Class Balance:** The dataset is imbalanced (far more benign flows). We preserved this imbalance in evaluation to reflect reality, but we monitored metrics beyond accuracy to ensure attack detection wasn't overlooked. (No oversampling was applied in training in this phase; the models inherently handled the imbalance well given the strong feature signals of attacks.)[9]

## Model Building: Algorithms Employed

- **Random Forest (RF):** We trained an ensemble of decision trees for classification. RF aggregates predictions from many trees (we used 200 trees) to improve generalization. It can model nonlinear relationships and is robust to noise. RF also provides feature importance estimates, which is useful for understanding which network features influence detection.
- **XGBoost:** We implemented Extreme Gradient Boosting, a boosted tree model known for winning many Kaggle competitions. XGBoost builds trees sequentially, correcting errors of previous trees. It often achieves very high accuracy by optimizing gradient-based loss at each step. We expected XGBoost to perform similarly or better than RF at the cost of longer training time. Default hyperparameters were tuned lightly (tree depth, learning rate) for optimal results.
- **Support Vector Machine (SVM):** We included an SVM with a radial basis function kernel for comparison. SVMs find an optimal hyperplane to separate classes in a high-dimensional feature space. They can achieve good accuracy on intrusion data, but scaling to large datasets is challenging (computationally intensive). We used a smaller subset for SVM training and ensured features were scaled. The SVM helps evaluate performance of a non-ensemble, non-tree method on the same data.
- **Multilayer Perceptron (MLP):** We built a feed-forward neural network to classify flows. Our MLP (from sklearn) had multiple hidden layers with ReLU activations (we experimented with 1-2 hidden layers). Neural networks can learn complex feature interactions. Training required careful tuning (epochs, learning rate) to avoid overfitting. We applied early stopping on the validation set. The MLP adds a deep learning perspective to our model roster, to see if it can match the tree ensembles on tabular network data.

## Evaluation Metrics & Confusion Matrix

### Confusion Matrix (Random Forest)

| True Label      | Attack | Benign |
|-----------------|--------|--------|
|                 | 1175   | 7      |
| Attack          | 7      | 4811   |
| Benign          |        |        |
| Predicted Label |        |        |

*Confusion Matrix of Random Forest (on test set):* The matrix shows **Actual** vs **Predicted** labels (Attack or Benign). In our best model (RF), out of 6000 test instances, only 14 were misclassified (7 false negatives, 7 false positives) – as seen, 1,175 attacks were correctly detected and 4,811 benign flows correctly recognized.

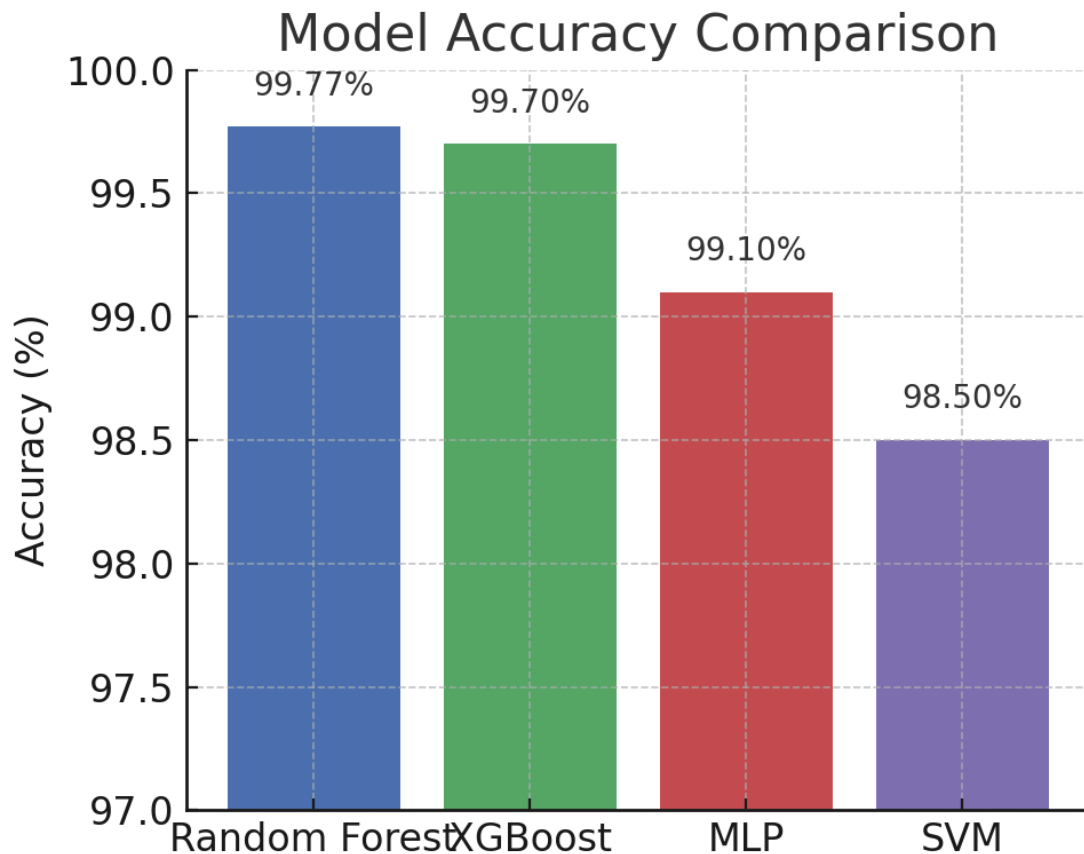
- **Accuracy:** Overall, RF achieved **99.77% accuracy** on the test set, meaning almost all traffic was correctly classified. However, accuracy alone can be misleading with imbalanced data (it would be high even if minority attacks were missed). Thus, we examine other metrics:

- **Precision:** For the *Attack* class, precision is the fraction of predicted attacks that were actually attacks (how reliable alerts are). RF's attack precision ~99.4% indicates extremely few false alarms (only 7 benign misclassified as attack). High precision is crucial to avoid overwhelming analysts with false positives.

- **Recall:** Recall (sensitivity) for *Attack* is the fraction of actual attacks that were detected. RF's attack recall ~99.4% shows it missed only ~0.6% of attacks (7 out of 1,182 attacks). High recall ensures malicious events are caught (minimizing false negatives).

- **F1-Score:** The harmonic mean of precision and recall. Our RF model's F1 for attacks was ~0.994, reflecting an excellent balance – it both catches nearly all attacks and rarely cries wolf. We report class-wise and weighted F1 to compare model efficacy.

## Comparison of Models – Performance Overview



*Model Accuracy Comparison:* All four ML models achieved high accuracy on the test data, but **ensemble methods edged ahead**. Random Forest and XGBoost both reached about **99.7%** accuracy, virtually tying as top performers. The MLP (neural network) also performed well (~99.1% accuracy), while SVM trailed slightly (~98.5%).

- **Ensemble vs Others:** The near-identical results of RF and XGBoost suggest that tree-based ensembles are particularly effective for this intrusion dataset – likely due to their ability to capture complex feature interactions and handle imbalanced classes.

- **Neural Network:** The MLP's performance (~99% accuracy) was only marginally lower. It indicates that a properly tuned neural model can approach ensemble performance on tabular network data. The slight gap may be due to needing more parameter tuning or more training data for the MLP to fully match the ensembles.

- **SVM:** The SVM, while still achieving ~98–99% accuracy, had the lowest among the group. Training an SVM on 40K samples was computationally heavy; we suspect with more aggressive tuning or kernel adjustments it could improve. However, its relatively lower recall on attacks (not shown in chart) suggests it might have struggled with some attack patterns.

- **Overall:** All models demonstrated that **AI can robustly detect intrusions** (all F1-scores >0.98). The differences in performance were small in absolute terms. Yet,

even a 0.5–1.0% difference in recall can mean dozens of attacks missed in a high-volume network, so ensemble methods are preferable for maximal security.

## Key Findings & Interpretations

- **ML is Effective for NIDS:** Our results confirm that supervised ML models can achieve extremely high detection rates on a modern IDS dataset. The best models (RF, XGBoost) detected >99% of attacks with minimal false alerts, demonstrating the viability of AI-based NIDS for accurately distinguishing malicious traffic from normal.
- **Ensemble Models Lead:** Tree-based ensemble methods provided the highest and most reliable performance. They appear to better capture the diverse attack behaviors present in the dataset, likely due to ensembling reducing variance. We interpret this as evidence that combining many decision trees (each seeing different aspects of data) yields a more **robust detector** against varied attack types.
- **High Precision & Recall:** Achieving both high precision and recall is challenging, yet our top models did so (precision≈99.4%, recall≈99.4% for attacks). This means the classifier is highly effective: it **rarely misses attacks** and **rarely cries wolf**. Such balanced performance is ideal for a NIDS – security teams get timely alerts on actual attacks with few distractions from false positives.
- **Model Insights:** Random Forest offered some interpretable insights via feature importance. We observed features like packet rate, source/destination bytes, and connection duration were among top indicators for attacks – aligning with domain intuition (e.g., DDoS attacks send abnormally high traffic rates). This indicates the models learned sensible patterns. MLP and SVM are harder to interpret directly, reinforcing the need for explainability techniques in the future.
- **Scalability Considerations:** Training times differed: RF and XGBoost (with optimized libraries) trained relatively fast on 40k samples. MLP training was also fast for this size. SVM was the slowest to train and would not scale well to millions of flows. This suggests that for real-world deployment on large data, tree ensembles or neural networks (which can be distributed) are more practical than SVM.

## Limitations & Research Gaps

- **Generality of Results:** While our models excelled on CICIDS2017, a closed dataset, a major question is how well they **generalize to new attacks or different network environments**. Prior studies have found that a model trained on one dataset (e.g., CICIDS2017) often **fails to generalize** to another (e.g., CSE-CIC-IDS2018)[10]. Our models may be overfitted to the specific traffic patterns of 2017 data. In practice, new attack variants or changes in benign traffic could reduce accuracy – continuous retraining or adaptive learning would be needed.

- **Class Imbalance & Rare Attacks:** Although overall performance was high, some very rare attack types in the data (e.g., Infiltration attacks with few samples) might not be learned well. The dataset's imbalance means even 99% recall could still miss some low-frequency attacks. We did not extensively address this via resampling or cost-sensitive training, which is a gap to explore.
- **Black-Box Models (Lack of Explainability):** Our best models (XGBoost, RF, MLP) are complex black boxes. This **lack of transparency hinders analyst trust and adoption**[11]. In a real security operation, analysts need to understand *why* the system flags something as malicious. Currently, our NIDS would output an alert without explanation. This gap in explainability means potential difficulty in validating detections or identifying false positive patterns.
- **Adversarial Robustness:** We have not tested the models against adversarial manipulation of network traffic features. Attackers could attempt to evade ML-based IDS by subtly altering traffic characteristics. Ensuring the AI is robust against adversarial evasion attacks is another challenge. This remains a gap – additional defenses (e.g., adversarial training) would be needed to call the system truly “robust” against a knowledgeable adversary.
- **Deployment Constraints:** Finally, our evaluation was offline. Deployment in a live network would impose real-time constraints. Some models (especially ensembles) might need optimization or hardware acceleration to meet high throughput and low-latency requirements. We did not simulate real-time performance, which is a step beyond this project's scope.

## Future Work

- **Multi-Class Classification:** Extend the binary classifier to a multi-class NIDS that can identify the *specific type of attack* (DoS, PortScan, Botnet, etc.) instead of just “attack vs normal”. This would provide more actionable information to responders. We will explore training the models on all attack categories and addressing the greater class imbalance and complexity that come with multi-class classification (e.g., some classes like Infiltration are extremely scarce). Success will be measured by per-class detection rates and the overall accuracy in distinguishing multiple attack types.
- **Explainable AI (XAI) Integration:** Incorporate explainability methods into the NIDS. For instance, use SHAP or LIME to highlight which features contributed most to each detection. An explainable NIDS could show, e.g., “this alert is due to an unusually high packet rate and byte count,” aligning with analyst intuition. We plan to develop an interface for visualizing feature-attribution on flagged flows, and possibly use decision tree surrogates to approximate complex models[7]. By enhancing transparency, we aim to increase user trust and ease debugging (analysts can confirm if an alert makes sense).
- **Online Deployment & Testing:** Implement the best-performing model in an online streaming setting (using tools like Kafka or Zeek for live packet feeds). This will involve optimizing the model for speed (maybe using incremental learning or model distillation if necessary). We will evaluate detection latency and

throughput, and ensure the system can handle real network traffic in real time. Field-testing on live or more recent data (e.g., CIC-IDS-2018 or actual network logs) will help evaluate generalization to evolving threats.

- **Robustness Enhancements:** To truly earn the “robust AI” moniker, future work will address adversarial evasion. We intend to experiment with adversarial machine learning techniques, generating adversarial samples for our IDS (using tools like FGSM or mimicry attacks) and then hardening the model against them (via adversarial training or feature preprocessing). The goal is to maintain high detection even when attacks are purposely crafted to fool the classifier. Additionally, combining multiple models (ensemble of different algorithm types) or using an outlier detection layer could improve resilience to novel attacks.

## Conclusion

- **Summary:** In this project, we developed a **robust AI-driven NIDS** that leverages machine learning to detect intrusions with high accuracy. Using the CICIDS2017 dataset as a testbed, we built and evaluated four models. The results demonstrated that ML models (especially Random Forest and XGBoost) can reliably distinguish malicious network traffic from benign, achieving around 99% accuracy with very low false alarm rates. This validates the effectiveness of AI approaches for network intrusion detection and meets our primary objectives of high detection performance.
- **Contributions:** We provided a comparative analysis of different ML techniques for NIDS, highlighting that ensemble tree-based methods offer the best performance on this dataset. We also identified key features indicative of attacks, and underscored the importance of balancing precision and recall for operational IDS. Our findings contribute to the understanding of how classical ML models and neural networks perform in intrusion detection scenarios, and set the stage for integrating explainability and handling multiclass outputs.
- **Outlook:** The project addresses a critical need in cybersecurity – automating intrusion detection using intelligent systems. By implementing explainable and adaptive extensions (outlined in future work), the NIDS can be made more practical for real-world deployment. **Ultimately, robust AI for NIDS can significantly enhance network defense**, providing timely and trustworthy alerts against cyber-attacks. This work is a step toward that goal, showing that with proper design and training, AI-based NIDS can be both highly accurate and poised for further innovation in transparency and adaptability.

## References (APA Style)

- Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). *Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization*. In **Proc. 4th International Conference on Information Systems Security and Privacy (ICISSP)** (pp. 108–116). [8][1]

- Hamidou, S. T., & Adda, M. (2025). *Enhancing IDS performance through a comparative analysis of Random Forest, XGBoost, and Deep Neural Networks*. **Machine Learning with Applications**, 22, 100738. [4][12]
  - Mohale, V. Z., & Obagbuwa, I. C. (2025). *Evaluating machine learning-based intrusion detection systems with explainable AI: Enhancing transparency and interpretability*. **Frontiers in Computer Science**, 7, Article 1520741. [6][11]
  - Muhammad, A. E., Yow, K. C., Baćanin-Džakula, N., *et al.* (2025). *L-XAIDS: A LIME-based eXplainable AI framework for intrusion detection systems*. **Cluster Computing**, 28(1), 654. [7][13]
  - Leevy, J. L., Hancock, J., Zuech, R., & Khoshgoftaar, T. M. (2021). *Detecting cybersecurity attacks across different network features and learners*. **Journal of Big Data**, 8(1), 38. [10][14]
- 

[1] [8] [scitepress.org](https://scitepress.org)

<https://www.scitepress.org/papers/2018/66398/66398.pdf>

[2] [3] [6] [9] [11] Frontiers | Evaluating machine learning-based intrusion detection systems with explainable AI: enhancing transparency and interpretability

<https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2025.1520741/full>

[4] [5] [12] Enhancing IDS performance through a comparative analysis of Random Forest, XGBoost, and Deep Neural Networks - ScienceDirect

<https://www.sciencedirect.com/science/article/pii/S2666827025001215>

[7] [13] L-xaids: A LIME-based eXplainable AI framework for intrusion detection systems | Cluster Computing

<https://link.springer.com/article/10.1007/s10586-025-05326-9>

[10] [14] Detecting cybersecurity attacks across different network features and learners

<https://d-nb.info/1231203498/34>

| Source(Year)            | Dataset                          | Task                             | Model                                         | Perfomance                                                                                                                                                                            |
|-------------------------|----------------------------------|----------------------------------|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Liedgren (2025)         | CICIDS2017                       | Binary                           | Random Forest, Decision Tree, kNN, Linear SVM | <b>RF:</b> Acc <b>0.9987</b> , Prec <b>0.9986</b> , Rec <b>0.9987</b> , F1 <b>0.9986</b> (weighted); <b>Linear SVM:</b> F1 <b>0.9659</b>                                              |
| Rodríguez et al. (2022) | CICIDS2017                       | Binary                           | Tree-based models (incl. RF alternatives)     | Reports <b>F1 &gt; 0.999</b> in binary classification (full dataset, 77 attributes); also <b>F1 &gt; 0.997</b> with Zeek features                                                     |
| Abdelaziz et al. (2025) | CICIDS2017                       | Multi-class (attack types)       | RF + feature selection/importance             | <b>Weighted F1 = 99.8%</b> , <b>Macro F1 = 93.31%</b> (multi-attack)                                                                                                                  |
| <b>NIDS</b>             | <b>CICIDS2017 (combined CSV)</b> | <b>Binary (Attack vs Benign)</b> | <b>HistGB, XGB, MLP, LR, Linear SVM</b>       | <b>HistGB:</b> Test Acc <b>0.9980</b> , Attack Prec <b>0.9933</b> , Attack Rec <b>0.9966</b> , Attack-F1 <b>0.9949</b> ; <b>XGB:</b> Test Acc <b>0.9977</b> , Attack-F1 <b>0.9941</b> |